

COMP2048: Theory of Computing

Dr. Kasra Khosoussi

Week 9

30/4/2026

The Class P

- Previous lecture:

$$P = \bigcup_k \text{TIME}(n^k)$$

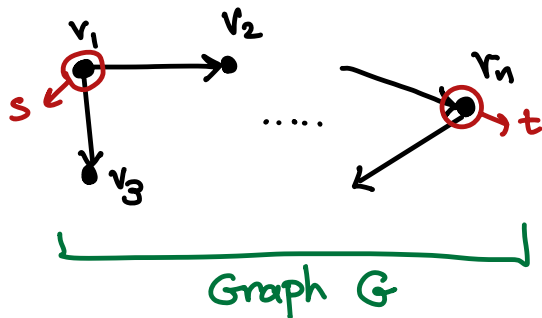
- P is the class of languages decidable in polynomial time (poly time) by a deterministic TM
- It captures decision problems that are efficiently solvable¹
- Many important/useful decision problems are in P
i.e., we have found poly-time algorithms² for solving them
- **Example:** given a graph G , is there a path from node s to t ?

$\text{PATH} = \{\langle G, s, t \rangle \mid G \text{ is a } \text{directed} \text{ graph with a path from } s \text{ to } t\}.$

¹Although, in theory, the exponent/constant can be very large ...

²Deterministic TMs

PATH



poly-time algorithm:

- mark s
- for every edge incident to a marked vertex
mark the other endpoint of edge
- if t is marked $\rightarrow$ accept
else $\rightarrow$ reject

- Many decision problems can be phrased as:

Is there an object satisfying a given property?

- **Example:** Is there a path from s to t in graph G ?
- These problems often have a natural brute-force algorithm:
 - list all candidate solutions
 - test each candidate one by one to see if it's a genuine solution
- **Example:**
 - list all sequences of vertices of length $|V(G)|$
 - for each sequence, check if first vertex is s , last vertex is t , and there's an edge between consecutive vertices in the sequence
- Number of candidates is often exponential in the input size
 $\Rightarrow$ brute-force algorithm usually takes exponential time
- Decision problems in P have a more clever algorithm that avoids searching through all candidates

Brute force for PATH

write all sequences of
of (at most n) vertices

$\nwarrow$
of vertices

$u_1 u_2 \dots u_k$



$O(n^n)$
many sequence

check

if $u_1 = s$

if $u_k = t$

if $(u_i, u_{i+1}) \rightarrow$ is an edge in G

But, we know that there is
a clever algorithm that runs in
poly time

- Many important decision problems have an extra property:
a genuine candidate solution can be checked in poly time

- **Examples:**

- ① PATH

$\{\langle G, s, t \rangle \mid G \text{ is a directed graph with a path from } s \text{ to } t\}$

- ② SAT

$\{\langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula}\}$

- ③ CLIQUE

$\{\langle G, k \rangle \mid G \text{ is an undirected graph with a } k\text{-clique}\}$

- ④ HAMPATH

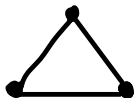
$\{\langle G, s, t \rangle \mid G \text{ is a directed graph w/ a Hamiltonian path from } s \text{ to } t\}$

- This is the idea behind class NP . . .

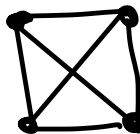
CLIQUE



clique
size 2

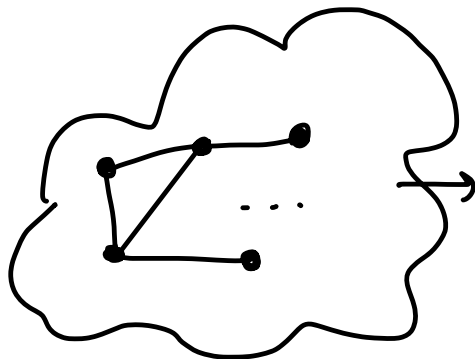


clique
size 3



clique
size 4

...



Graph G

does it have
a size- k clique?

SAT

- A Boolean formula is built from variables $(x_1, x_2, \dots)$ and connectives (AND, OR, and NOT):



- Each variable is either TRUE or FALSE
- A formula is **satisfiable** if some truth assignment makes it TRUE

Definition

$$\text{SAT} = \{ \langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula} \}$$

- Example:

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

is satisfiable (e.g., $x_1 = \text{TRUE}$, $x_2 = \text{TRUE}$, $x_3 = \text{TRUE}$)

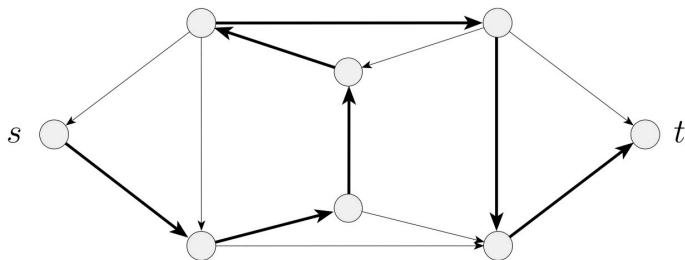
- Example:

$$x \wedge \neg x$$

is not satisfiable

HAMPATH

Does G have a path from s to t s.t. the path visits every vertex exactly once



NP: Nondeterministic Polynomial Time

- There are two equivalent ways to define NP:
 - ① Via polynomial-time verifiers
 - ② Via **nondeterministic** Turing machines
- Either definition can be taken as primary; one can prove they define the same class of languages

Polynomial-Time Verifiers

- A **verifier** for a language L is deterministic TM V such that

$$L = \{ w \mid \exists c \in \Sigma^* : V \text{ accepts } \langle w, c \rangle \}$$

$w \in L \Leftrightarrow$ there exists a string c such that V accepts $\langle w, c \rangle$

- c is called a **certificate** or **proof** (of membership in L)
- A poly-time verifier for L is a verifier such that:
 - $|c| \leq p(|w|)$ for some polynomial p
 - V runs in poly time in $|w|$

Definition

NP is the class of languages that have a poly-time verifier.

Examples of languages in NP

● PATH

- input: graph G and vertices s and t
- certificate: a path from s to t

● SAT

- input: Boolean formula ϕ
- certificate: a truth assignment that satisfies ϕ

● CLIQUE

- input: graph G and integer k
- certificate: k vertices that form a clique

● HAMPATH

- input: graph G and vertices s and t
- certificate: a Hamiltonian path from s to t

test $\langle \phi \rangle \in \text{SAT}$

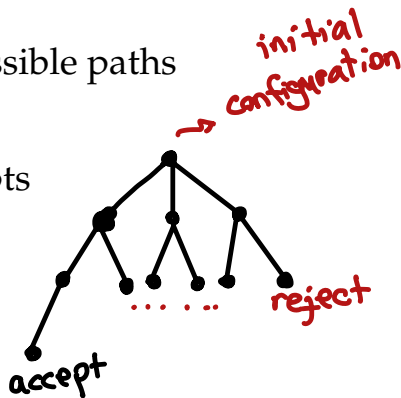
if $\langle \phi \rangle \in \text{SAT}$
↓
 $x_1 = \text{FALSE}$
 $x_2 = \text{TRUE}$
⋮
 $x_n = \text{FALSE}$ C

In each case:

- certificate length is polynomial in size of input
- verification is polynomial-time in size of input

Nondeterministic Turing machines (NTM)

- A nondeterministic TM (NTM) may have several possible moves from one configuration
- A computation can branch into many possible paths
- The machine **accepts** if *some* branch accepts
- The machine **rejects** if *all* branches reject
- An NTM is a decider if all branches halt



Time for nondeterministic machines

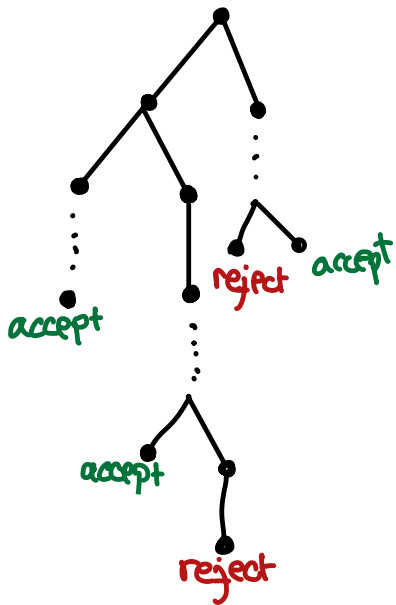
Definition

Let M be an NTM decider. Its running time on inputs of length n is the maximum number of steps used on any branch, over all inputs of length n .

Definition

$$\text{NTIME}(f(n)) = \{ L \mid \text{some NTM decides } L \text{ in time } O(f(n)) \}$$

- This is the nondeterministic analogue of $\text{TIME}(f(n))$



non-deterministic
run time

Deterministic
TM



Second definition of the class NP

Definition

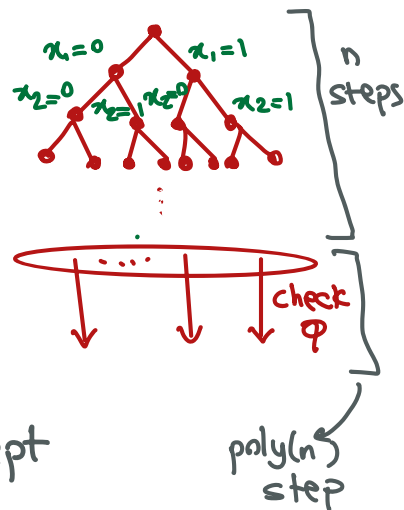
NP is the class of languages decidable in (nondeterministic) polynomial time by a nondeterministic Turing machine.

$$\text{NP} = \bigcup_k \text{NTIME}(n^k)$$

- NP stands for **Nondeterministic Polynomial time**
- This is equivalent to the verification definition of NP

Example: SAT

- nondeterministically^y enumerate (write onto tape) all possible true/false assignments variables
- check each assignment
- if finds a truth assignment that satisfies $\phi \Rightarrow$ accept
else reject.



P and NP

P: languages where membership can be *decided* (tested) efficiently

NP: languages where membership can be *verified* efficiently

- $L \in \text{NP}$ does *not* mean L is “hard” (not solvable in poly-time)
- In fact, $P \subseteq \text{NP}$ (why?)
i.e., all efficiently decidable languages are in NP
- We don't know if $P = \text{NP}$ or $P \neq \text{NP}$
- No poly-time algorithm is known for many problems in NP (e.g., HAMPATH, SAT, etc)
- We are unable prove existence of language in NP that is not in P

Why $P \subseteq NP$

From the verification definition of NP:

- Suppose $L \in P$
- Then there is a deterministic polynomial-time decider for L
- A verifier can simply:
 - ignore the certificate c , and
 - run the polynomial-time decider on the input
- Therefore,

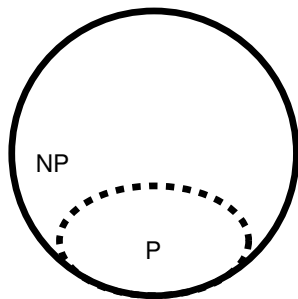
$$P \subseteq NP$$

From the NTM definition of NP:

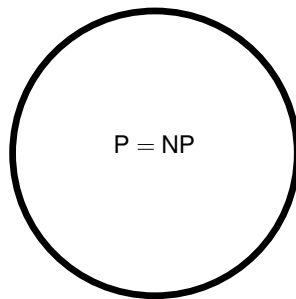
- Suppose $L \in P$
- Then there is a deterministic polynomial-time TM decider for L
- Any deterministic TM is an NTM (that simply doesn't use branching/nondeterminism)
- Therefore,

$$P \subseteq NP$$

Two Possible Worlds



$P \neq NP$



$P = NP$

most computer scientists believe $P \neq NP$

Polynomial-time many-one reduction

Definition

We say language A is poly-time reducible to language B

$$A \leq_p B$$

if there exists a polynomial-time computable function^a f such that

$$x \in A \iff f(x) \in B$$

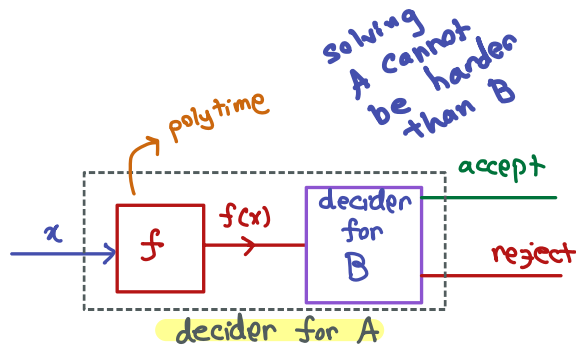
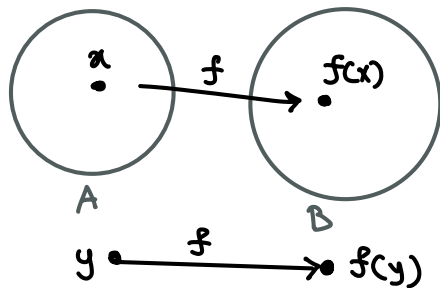
for every input x .

^asee Definition 7.28 in Sipser for a formal definition

- One instance of A (i.e., testing if $x \in A$) is transformed (“translated”) into one instance of B (i.e., testing if $f(x) \in B$)
- This reduction (i.e., evaluating f) must be doable in poly-time
- Then we call the decider for B on $f(x)$
- If $f(x) \in B$, then $x \in A$, and otherwise if $f(x) \notin B$, then $x \notin A$

How a reduction works

$$x \in A \Leftrightarrow f(x) \in B$$



Key theorems about reductions

In the next Applied Class or Assignment 3, you'll prove:

Theorem

If $A \leq_p B$ and $B \in P$, then $A \in P$.

Theorem

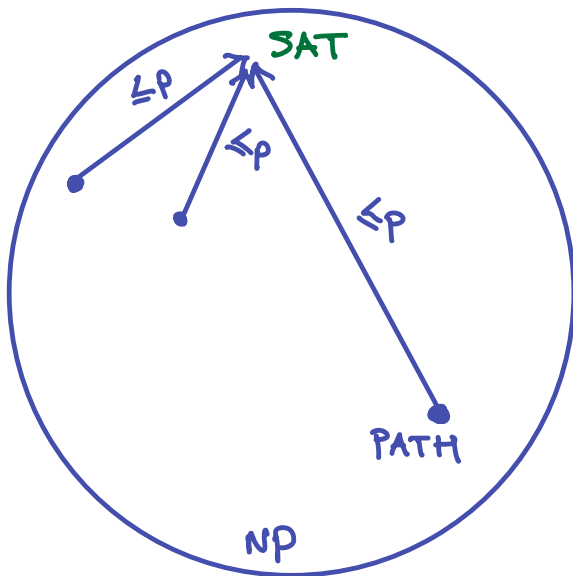
If $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.

The Cook–Levin Theorem

Theorem (Cook–Levin)

$$L \leq_p \text{SAT} \quad \forall L \in \text{NP}$$

- We already showed that $\text{SAT} \in \text{NP}$
- Cook–Levin says that every language in NP can be reduced in polynomial time (by a deterministic TM) to SAT
- SAT is at least as “hard” as any other problem in NP
- If one can find an algorithm for solving SAT in polynomial time, we can solve every problem in NP in polynomial time (why?)



NP-hard and NP-complete

Definition

A language B is **NP-hard** if for every language $A \in \text{NP}$,

$$A \leq_p B$$

Definition

A language B is **NP-complete** if:

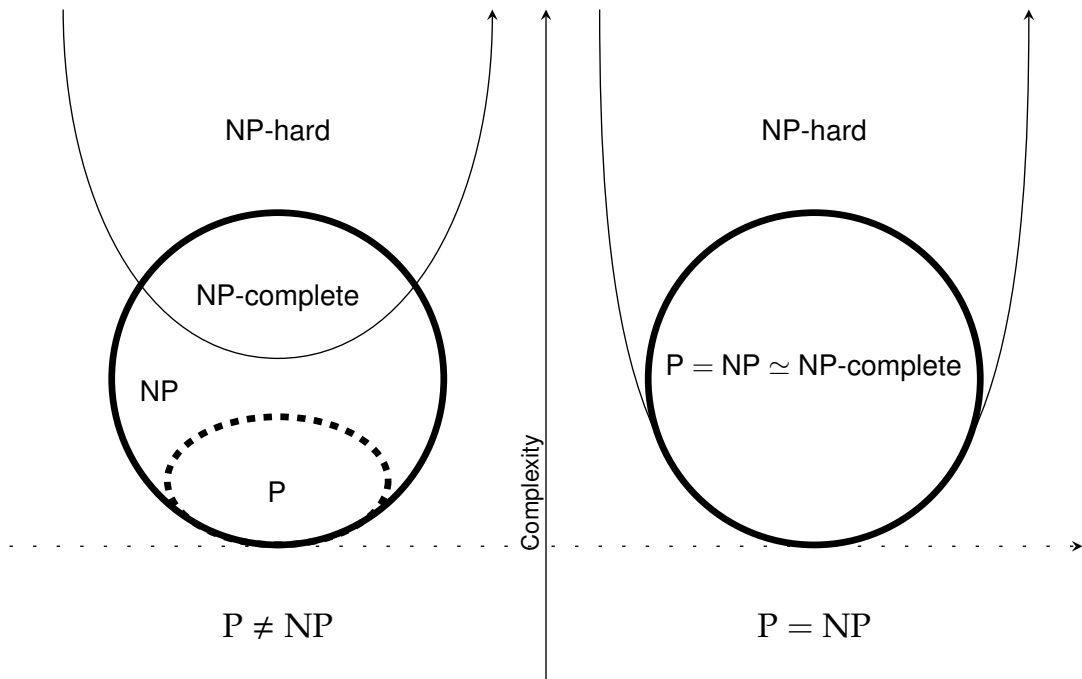
- $B \in \text{NP}$, and
- B is NP-hard

- NP-complete problems are the “hardest” problems in NP
- If any NP-complete problem is in P, then

$$P = \text{NP}$$

- If some NP-complete problem is proved not to be in P, then

$$P \neq \text{NP}$$

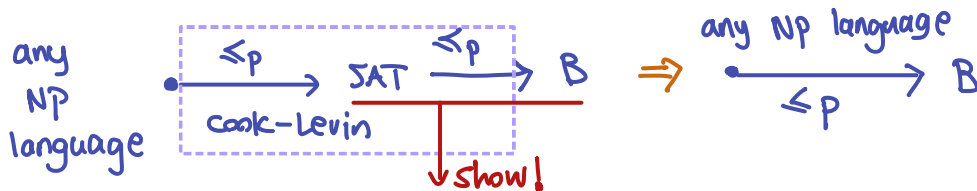


How to prove a problem is NP-complete

- From Cook–Levin, we know SAT is NP-complete
- We now know thousands of NP-complete problems
- SAT, HAMPATH, CLIQUE, ...
- To prove that a language B is NP-complete:
 - ① show that $B \in \text{NP}$
 - ② choose a known NP-complete problem A (e.g., SAT, HAMPATH)
 - ③ prove

$$A \leq_p B$$

Direction of $\leq_p$ matters! You'll make a mistake at least once



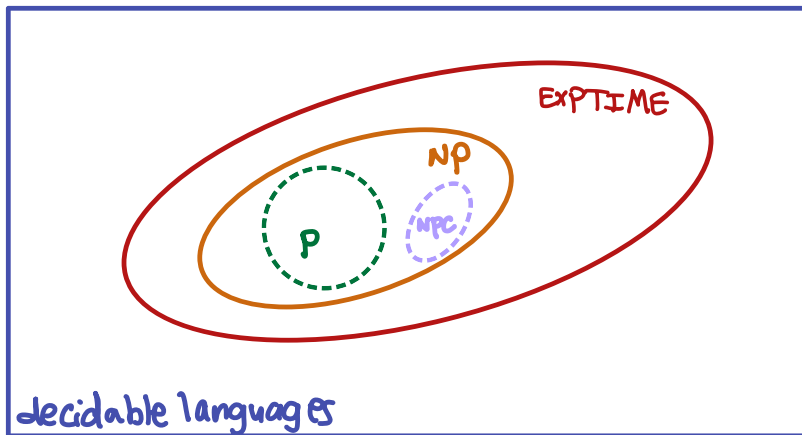
EXPTIME

Definition

$$\text{EXPTIME} = \bigcup_{k \geq 1} \text{TIME}(2^{n^k})$$

- Class of languages decidable in deterministic exponential time
- $\text{NP} \subseteq \text{EXPTIME}$ (proof idea: try all poly-size certificates!)
- We know there are problems in EXPTIME that are not in NP

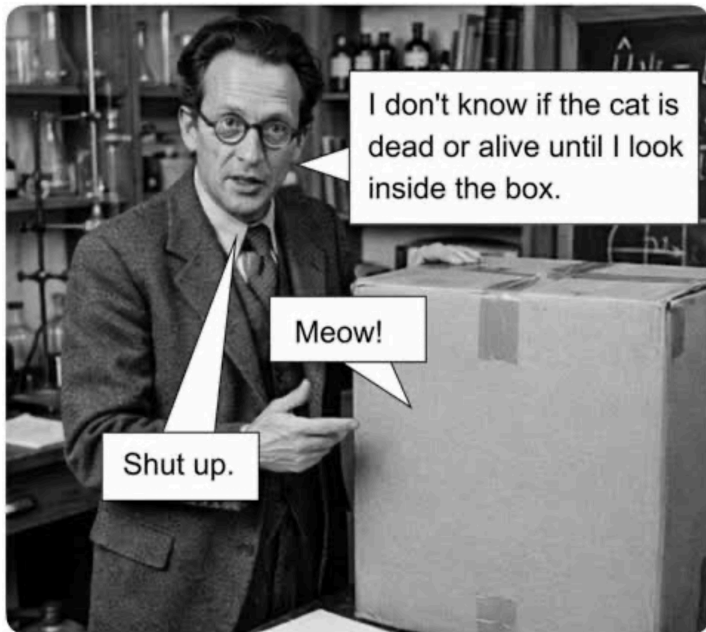
Time Complexity Big Picture



$$P \subseteq NP \subsetneq EXPTIME$$

Common pitfalls

- NP does not mean “not polynomial.”
- NP does not mean “known to be hard.”
- $A \leq_p B$ means: an efficient algorithm for B gives one for A .
- To prove B is NP-complete, reduce a known NP-complete problem *to* B .



I don't know if the cat is dead or alive until I look inside the box.

Meow!

Shut up.